

Technical Note #1

Design Trade-offs and Failure Modes in a Real-Time Market Data Pipeline

Charles Colella
contact@charlescolella.com

May 2026

Abstract

This technical note studies a real-time market-data pipeline that ingests incremental updates from multiple exchanges, reconstructs per-symbol order books, and computes derived streams online. The note analyzes how such a system remains operational under irregular input, throughput constraints, and saturation. It combines pipeline design, implementation details, and a live illustration covering 934 order books and more than 280 million output records.

1 Introduction

When a pipeline processes real-time streams, it must produce incremental results from continuously arriving data, without waiting for the input to become fully stable. Yet in timeliness-sensitive systems, preserving strong processing guarantees may conflict with bounded delay and backlog.

Market-data processing is a direct example of this problem. A market-data pipeline receives updates from multiple venues, reconstructs per-symbol order books, and computes derived streams online. The input is bursty and uneven across venues and symbols, especially during the market open. The system must continue to emit usable outputs while handling recovery, discontinuities, and pressure on compute resources.

This note is the first in a technical series on a distributed real-time market-data system. It focuses on the design and implementation of the pipeline, and on its behavior under changing load around the U.S. market open. The system ingests incremental updates from multiple exchanges, reconstructs per-symbol Level 2 order books, and computes derived streams online. The application code, infrastructure configuration, and deployment materials used to run the live pipeline are provided with the study [4].

The analysis follows three levels. First, we describe the pipeline design and the choices used to keep the system live under pressure. Second, we describe the deployed implementation and the mechanisms used to handle irregular input, recovery, and saturation. Third, we illustrate the behavior of the system on a bounded live window covering 934 order books and more than 280 million output records.

The second note builds on the same setting and studies how the pipeline transforms the timing of the streams it processes, and how those temporal transformations can affect downstream signals [3].

2 System Design

This section describes the system design. Each component is presented in three parts. Role and contract give the local semantics. Internal mechanism covers the mechanisms that shape ordering, loss, and backlog. Failure boundary covers behavior under saturation or failure.

2.1 Overview

The system is a real-time market data pipeline. It ingests incremental exchange updates, reconstructs per-symbol Level 2 order books, and computes derived feature values from successive order book states. Its input consists of asynchronous update streams from multiple exchange-symbol pairs. Its output consists of bounded-depth order book states and derived feature streams produced online from those states. The basic processing unit is the exchange-symbol stream, and the pipeline treats these streams independently.

The main body of this document focuses on the hot path. This is where exchange updates are received by the *feed handler*, buffered by the *transport layer*, reconstructed into deep Level 2 order books by the *order book engine*, and transformed into derived feature values by the *feature extractor*. Outside the hot path, derived outputs are sent through a downstream *transport layer* to the *feature store* for storage and offline analysis. Snapshot requests follow a separate path through the *snapshot middleware*, which mediates recovery requests with the upstream exchange. These additional components are described in the appendices. Figure 1 summarizes the full system and highlights the hot path.

Upstream of the order book engine, no strict processing guarantee is required. For each exchange-symbol pair, the system maintains only a temporary local reconstruction of the order book. When recovery is needed, the exchange is used as the authoritative external source, and in this architecture rebuilding from a fresh snapshot is preferred to replaying a large update history. The hot path therefore does not re-

bounds are reached, publication stops draining additional events until capacity becomes available again. Downstream pressure is therefore exposed as explicit saturation to avoid unbounded local backlog growth.

Transmission remains best-effort. On a retrievable failure, a record is returned to its partition retry buffer and sent again after backoff. In rare cases, especially under transient network instability, a later record may already have left the queue before the failure is detected, so the retried record may be accepted after it. Occasional within-partition reordering is therefore treated as an expected anomaly and is handled during downstream reconstruction.

2.3 Transport Layer

2.3.1 Role and contract

The transport layer is the buffering and delivery layer between ingestion and downstream processing. It stores normalized records, makes them available to consumers, and decouples publication from consumption.

Its local contract is: (i) For a given exchange-symbol pair, records are consumed in the same order in which they are published. (ii) Producers and consumers progress asynchronously. (iii) Consumption remains independent from short-term publication fluctuations. (iv) Writes are best-effort only: records may be lost under failure before replication completes.

2.3.2 Internal mechanism

The transport layer is organized as a fixed set of partitions. Each partition stores one in-order append-only stream of records. Multiple exchange-symbol streams may be mapped to the same partition, and multiple partitions may be mapped to the same transport instance. Figure 4 summarizes this structure.

Producer and consumer progress remain decoupled. On the write side, records are first received and queued, then appended locally before an acknowledgement is returned to the producer. Consumers read later and advance at their own pace. The transport layer therefore acts as an independent buffering zone between ingestion and downstream processing. The trade-off is that, if consumption remains slower than production, consumer lag can grow without bound.

Persistence remains local first, then replicated. Records are persisted locally on disk and replicated across 2 transport instances, so that a secondary instance can take over if the primary one fails for a given partition. Within each partition, data is stored in binary segment files and appended sequentially as new records arrive. Each partition therefore keeps its own persistent local stream. Replication to the secondary instance remains asynchronous and takes place after the local write.

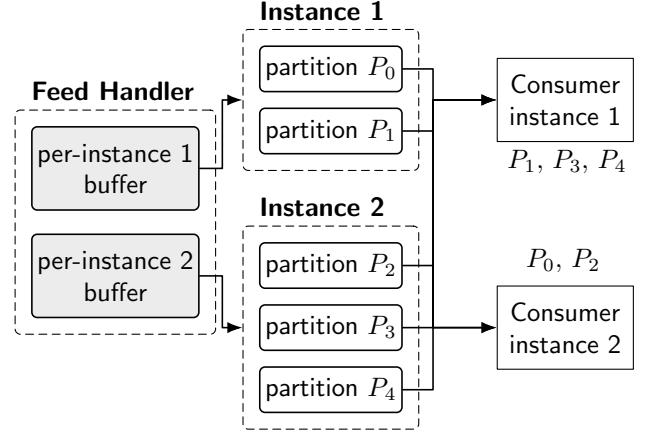


Figure 4: High-level structure of the transport layer. The example shows 5 partitions distributed across 2 instances where records are written by a single producer and read by 2 consumers.

2.3.3 Failure boundary

Internal network and write buffers are bounded. Under sustained input pressure, they may saturate and propagate backpressure upstream through increasing request latency and eventual request timeouts.

Replication remains asynchronous. Records are acknowledged after the local write, so a failure before replication completes may still cause recent records to be lost.

2.4 Order book engine

2.4.1 Role and contract

The order book engine reconstructs and maintains Level 2 order books in real time. It consumes incremental depth updates from the transport layer, applies them per symbol, and produces a bounded-depth order book state for downstream use.

Its local contract is: (i) Order book state is maintained independently for each exchange-symbol pair, and updates are applied in exchange sequence order. (ii) Local state may be lost under failure, but each such loss is detected explicitly. (iii) When local consistency cannot be preserved, the current state is discarded and recovery is triggered.

2.4.2 Internal mechanism

The order book engine is organized into two distributed layers. The consumption layer fetches update records from the transport layer and forwards them to the processing layer. The processing layer maintains the local order book state for each symbol. Exchange-symbol pairs follow a many-to-one mapping onto processing instances, so all updates for a given pair are routed to the same local order book. This separation keeps transport consumption independent from stateful reconstruction. Figure 5 summarizes this execution model.

State updates must remain sequential and locally recoverable. For a given exchange-symbol pair, updates are decoded, applied to the current state, and turned

into a bounded-depth Level 2 output. For each local book, the exchange-provided sequence number defines the only valid application order. A small bounded re-order buffer is kept to absorb rare out-of-order arrivals. If the missing sequence numbers are recovered before the buffer limit is reached, reconstruction continues locally. If they do not, the current state is treated as invalid and a fresh snapshot is requested from the corresponding exchange.

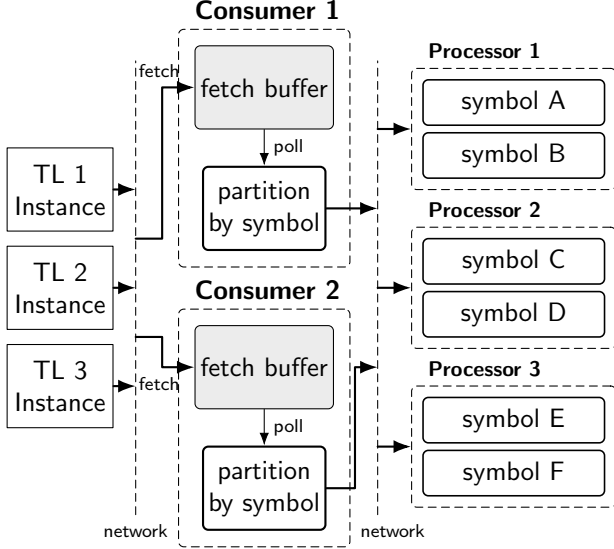


Figure 5: Internal architecture of the order book engine. The example shows 3 transport-layer instances, 2 consumers, and 6 exchange-symbol pairs distributed across 3 processing instances.

Snapshot recovery should not interrupt processing. Incoming updates therefore continue to be buffered in sequence order. This allows the engine to detect new sequence breaks without delay and preserves the update order needed to apply the snapshot consistently once it arrives.

Snapshot requests must also remain infrequent. The engine therefore maintains a deeper local book than the bounded depth it publishes downstream. This is mainly useful for exchange-symbol pairs whose prices move quickly across levels. For those books, the published depth is sufficient for downstream use, but too narrow to maintain stable local reconstruction. The extra local depth therefore acts as a stability margin.

2.4.3 Failure boundary

If processing time becomes too large relative to the incoming flow, pressure propagates upstream to the associated consumer, which reduces its fetch activity.

When a processing instance fails, its local order book state is lost. Recovery then relies on a fresh snapshot from the exchange rather than on local state replay. Snapshot recovery remains asynchronous and may take longer than the incoming update flow. The recovery path is therefore guarded by explicit limits. If waiting time becomes too long, or if out-of-sync events recur too often before recovery completes, the associated or-

der book is treated as unstable and moved to an inactive state for manual intervention.

2.5 Feature extractor

2.5.1 Role and contract

The feature extractor consumes successive order book states to compute derived feature values.

Its local contract is: (i) Order book updates are processed in order for each symbol. (ii) Each input update produces at most one output per feature.

2.5.2 Internal mechanism

Per-pair processing must remain local and ordered. Feature computation is therefore distributed across multiple extractor instances. Exchange-symbol pairs are mapped many-to-one onto these instances, so all updates for a given pair are routed to the same one. Figure 6 summarizes this execution model.

Feature computation may be stateless or stateful, but it remains incremental. In both cases, each received update produces one feature value per feature, and outputs are published independently.

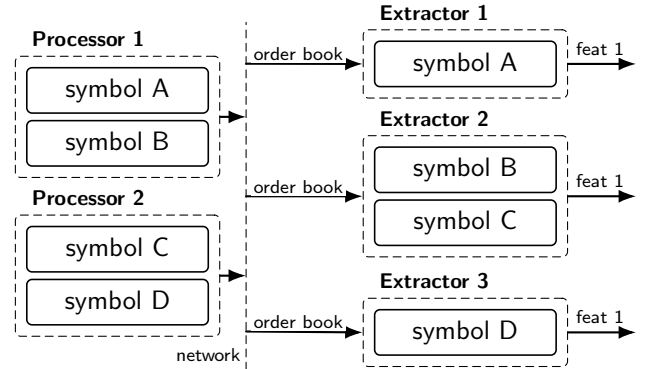


Figure 6: Internal architecture of the feature extractor. The example shows one feature computed on 4 symbol order books across 3 processing instances.

2.5.3 Failure boundary

If the local output rate becomes lower than the input rate, the input buffer may saturate. This saturation then propagates upstream to the order book engine for the corresponding exchange-symbol-feature path.

Publication remains non-persistent. If an extractor instance fails, queued or in-flight updates are lost and are not replayed. Any local feature state maintained by that instance is lost as well.

3 Implementation

This section describes the concrete implementation of the system architecture. Each component is presented in two parts: the implementation choices behind its main mechanisms, and the deployment settings used here.

3.1 Overview

The implementation relies on a small set of mature distributed systems and runtimes. Kafka is used for stream transport, while Flink is used for stateful online processing. Low-latency ingestion is implemented in Rust with Tokio, and inter-component messages are exchanged in binary form. The whole pipeline runs online on a shared cluster, with logical separation between exchanges and between the main processing stages.

The deployment described here covers two public spot exchanges, Binance and KuCoin. Binance publishes incremental updates at a fixed 50 ms cadence, while KuCoin publishes incremental updates as they occur. These two feeds therefore expose different update regimes and different load profiles. The system processes all symbols listed simultaneously on both exchanges at the time of writing, for a total of 467 exchange-symbol pairs per exchange and up to 934 concurrent order books across the full pipeline.

KuCoin produces a denser and more irregular flow than Binance, so its processing path is provisioned with more parallelism. Across all symbols, the input rate is roughly 5k to 50k updates per second per exchange, depending on market activity. KuCoin typically emits one update per event, whereas Binance aggregates updates over 50 ms windows. Load is also highly heterogeneous across symbols.

3.2 Feed Handler

The feed handler is implemented in Rust and runs on the `tokio` asynchronous runtime to handle multiple exchange streams concurrently within a single instance. One instance is deployed per exchange. On the publication side, records are sent to the transport layer through Kafka, discussed in Section 3.3. This producer path is implemented with `librdkafka`. On the ingestion side, network I/O uses the low-overhead socket interface `fastsocket`.

In the deployment described here, one instance is deployed per exchange and ingests 467 spot symbols through WebSocket feeds. The Binance instance uses the SBE feed [1], with a 50 ms update cadence, and the KuCoin instance the Pro feed [6], with real-time updates. These were the highest-cadence public feeds available at the time of writing. Parallelism is fixed at the task level, with 300 stream connection tasks on the ingestion side and 467 publication tasks on the publication side. Symbols are assigned deterministically from recent trading activity so that the most active ones are not colocated. Local bounds are also fixed, with inter-plane queues limited to 300 entries and publication limited to 30 queued records per task. The transmission path remains bounded through capped batching, capped request size, and limited retries.

3.3 Transport Layer

The transport layer is implemented with Apache Kafka [5] as a distributed append-only log. Partition data is stored on local SSDs as append-only segment

files.

In the deployment described here, the transport layer runs on a 3-instance cluster with 20 partitions per exchange and a replication factor of 2, or about 23 exchange-symbol streams per update partition under an even distribution. The same infrastructure is shared by the main update path, snapshot delivery, and downstream feature publication, while partition placement remains fixed and automatic rebalancing is disabled. The transport layer is used here as a short-term buffer, not as the main recovery store. Recovery falls back to fresh exchange snapshots, so retention is kept short after segment rotation, typically 3 to 5 minutes. Log files are rotated every 60 minutes with an additional random jitter of up to 45 minutes to avoid synchronized roll events across partitions. Persistence follows the operating-system write-back policy, independently of log rotation.

3.4 Order book engine

The order book engine is implemented with Apache Flink [2] as a distributed stateful processing system on the JVM. Order book state is kept in memory through Flink’s heap-based state backend. The engine follows an explicit three-state model: *in_sync*, *out_of_sync*, and *idle*. In the *in_sync* state, incremental updates are applied continuously and order book outputs are produced for each event. In the *out_of_sync* state, the current local state has been invalidated and snapshot-based recovery is in progress. In the *idle* state, repeated recovery failure has led to a stop that requires manual intervention. The engine does not use persistent local state for recovery, *i.e.* no checkpoint-based recovery in Flink. After a failure, it resumes from the current end of the transport partitions and rebuilds local state from a fresh exchange snapshot. This design prioritizes rapid recovery over replaying missed events, at the cost of potentially skipped outputs.

In the deployment described here, the Flink cluster runs in session mode, with one job per exchange. Parallelism and task placement are fixed at startup, and no automatic rescaling or rebalancing takes place during operation. For Binance, the job runs with 2 consumers and 2 processors. For KuCoin, it runs with 5 consumers and 9 processors. The order book engine is deployed on 8 compute instances shared with the feature extractor, with logical isolation between processing paths. Up to 3 events per symbol may be held temporarily before sequence continuity is restored. Only the top 50 levels per side are exposed in the output order book, while up to 1000 levels per side are maintained internally. Recovery is also bounded by a fixed timeout and by a 10k-event recovery buffer. If repeated snapshot attempts still fail to restore a consistent state, the engine transitions to *idle*.

3.5 Feature Extractor

The feature extractor is implemented with Apache Flink [2] and runs directly downstream of the order book engine in the same job for each exchange. It

computes a small set of online microstructure features from successive order book states, including mid-price, log return, spread, micro-price, depth around the mid-price, and multi-level imbalance. Some of these features depend only on the best bid and ask, while others aggregate liquidity across several order book levels. Formal definitions are provided in Appendix A.5.

In the deployment described here, feature computation remains logically isolated for each exchange-symbol pair. Feature computation is divided into 3 groups to use compute resources efficiently, with parallelism 1 per group for Binance and 4 per group for KuCoin. The component shares 8 compute instances with the order book engine, with logical isolation between processing paths.

4 Observed behavior

This section presents a small set of live system observations. It focuses on the main operational signals of the pipeline across a regime change and makes visible some of the effects of the architectural choices described earlier. The interpretation remains limited to this window. Additional observations on resource usage, network activity, and output timing are provided in Appendix A.6.

4.1 Conditions and methodology

The observations in this section are interpreted against three expectations: (i) end-to-end latency increases with input message rate, (ii) queueing delay dominates latency under high load, and (iii) message loss correlates with buffer saturation events. End-to-end latency is measured from message reception in the feed handler, before application-level processing, to feature emission after computation.

The system is observed on live data from both Binance and KuCoin over a one-hour measurement window spanning March 3, 2026 from 14:00 to 15:00 UTC. The observation covers 467 symbols per exchange, restricted to the symbols listed simultaneously on both venues. Over this one-hour window, the pipeline produced about 280 million output records, or roughly 58 GiB of data. The system runs on a cluster provisioned with 104 vCPU and 416 GiB of RAM across 8 nodes. Latency values below $1326\mu s$ are not interpreted, according to Appendix A.4. Operational signals are scraped centrally every 10 seconds from all components and interpreted a posteriori. Additional deployment material is described in Appendix A.3, and implementation details are available in the public repositories [4].

4.2 Global pipeline behavior

Figure 8 reports the aggregate message rates observed at the pipeline input and output across all symbols. A clear transition occurs at 14:30 and separates two market activity regimes. Before 14:30, activity remains comparatively moderate and stable. After 14:30, it increases markedly and exhibits more frequent spikes.

This transition coincides with the start of the main U.S. cash-equity trading session. The selected window therefore provides two distinct operating regimes for illustrating system behavior under changing load.

Figure 8 also gives an order of magnitude of the data made available by the two exchanges during this window. This is the flow seen by the pipeline. Input and output rates follow the same regime change, while output throughput remains higher because each accepted order book update produces 6 feature records.

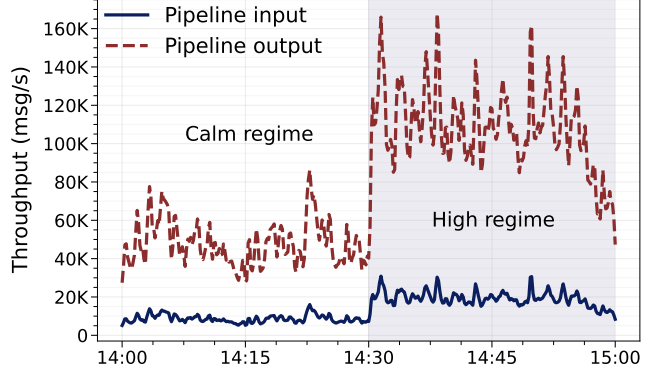


Figure 8: Aggregate input and output message rate for Binance and KuCoin, 467 symbols each, computed over a 30s sliding window.

Exchange	Calm regime	High regime	Total
KuCoin	70.7M	165.6M	236.4M
Binance	15.7M	25.1M	40.8M

Table 1: Feature counts observed at the pipeline output.

Table 1 also reflects two distinct input stream structures. Binance aggregates updates into less frequent events, while KuCoin emits a more granular real-time stream with lighter events. The contrast is especially visible on the most active symbols. For BTC-USDT, for example, Binance events contain about 40 updates on average in the calm regime and about 120 in the high regime. On KuCoin, the same pair remains close to one update per event. The overall event rate is therefore driven primarily by KuCoin. This makes it possible to observe how the pipeline reacts under two distinct input stream structures.

Figure 7 shows that these two input stream structures do not stress the pipeline in the same way. On Binance, both average latency and p95 remain contained and stable, and are only weakly affected by the change in load regime. On KuCoin, both metrics rise more clearly in the high regime and show stronger variability, with repeated spikes that are amplified in p95. This suggests that latency variability is driven more by message rate than by message size. The stronger spikes observed on KuCoin point more to queueing effects and short-lived backlog accumulation than to processing cost alone. In this system, saturation therefore appears to be more sensitive to message rate than to message size, at least over the regimes observed here.

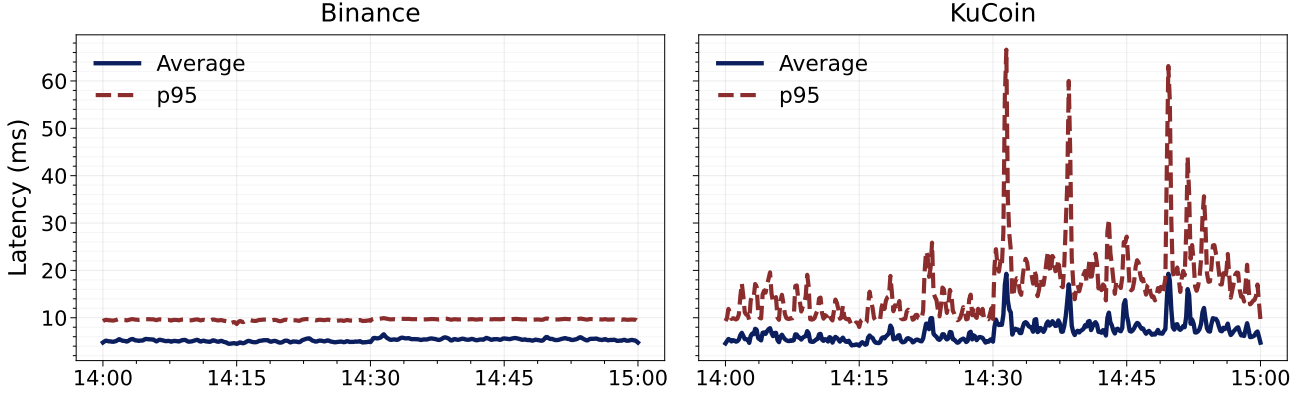


Figure 7: End-to-end pipeline latency from message reception to the end of feature extraction, computed over a 30s window.

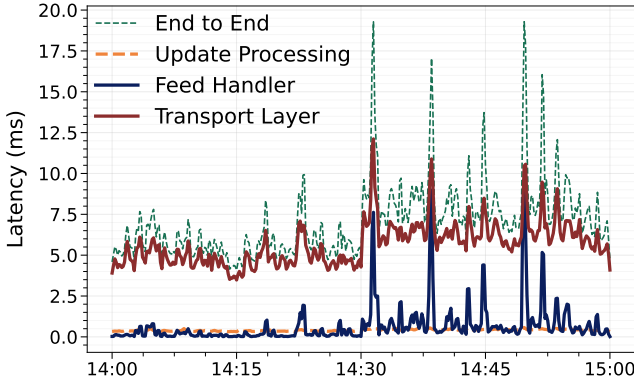


Figure 9: Average latency of each stage on the KuCoin hot path, computed over a 30s sliding window.

Figure 9 shows that queuing effects accumulate across stages on the KuCoin path. The latency spikes coincide in the *feed handler* and the *transport layer*, and together they shape most of the end-to-end latency. Short-lived bursts first accumulate in the feed handler and then propagate into the transport layer, which acts as the main buffering boundary. As a result, processing in the order book engine and the feature extractor remains small relative to the total latency. For KuCoin, latency peaks therefore appear to be shaped primarily by cumulative queuing delays before processing.

4.3 Saturation and resynchronization

Queueing effects can also directly affect outputs by turning delay into message loss and snapshot recovery. Figure 10 reports the time spent in the inter-plane queue together with `buffer_full` events for KuCoin. The delays observed there also coincide closely with those seen on the feed-handler end-to-end path. Saturation appears only during the most stressed periods, close to the largest latency peaks. This waiting behavior follows directly from the architectural choice to process each exchange-symbol pair sequentially. Under a burst, the system must catch up in order, so waiting time increases until the queue limit is reached. Bursts are first absorbed as delay, then converted into explicit loss once the limit is reached.

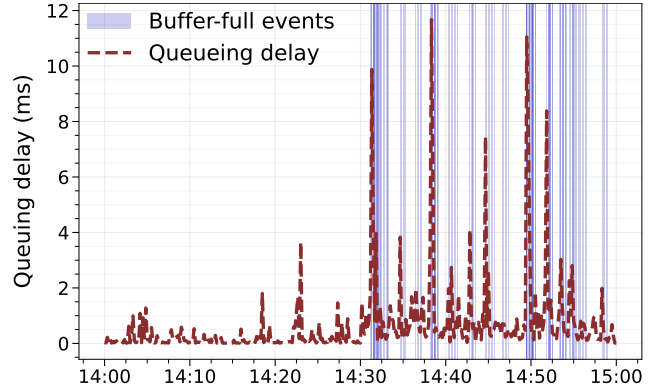


Figure 10: Average inter-plane queuing delay over time for KuCoin, with `buffer_full` events indicating explicit saturation, computed over successive 30s windows.

This behavior matches the mechanism described in Section 2.2.3. The effect is especially visible on KuCoin, where the feed is more granular. The resulting pressure remains heterogeneous across symbols and over time. Binance is omitted for readability, as no saturation is observed there.

Each explicit loss upstream translates into snapshot recovery downstream. Figure 11 reports the fraction of active symbols in recovery over time. Recovery remains infrequent overall, but its peaks follow the same stressed periods as queue saturation. On KuCoin, for example, 21 snapshot recoveries were triggered for ETH-USDT by sequence breaks, against 1 triggered by price movement beyond the maintained depth. For BTC-USDT, the corresponding counts are 12 and 3. Upstream pressure therefore reaches the output: once loss occurs, output stops until recovery completes.

Each explicit loss upstream translates into snapshot recovery downstream. Figure 11 reports the fraction of active symbols in recovery over time. Recovery is observed on both exchanges, but for different reasons. On KuCoin, recovery peaks follow the same stressed periods as queue saturation and are dominated by sequence breaks. On Binance, where no queue saturation is observed, recovery is instead dominated by *limit reached* events.

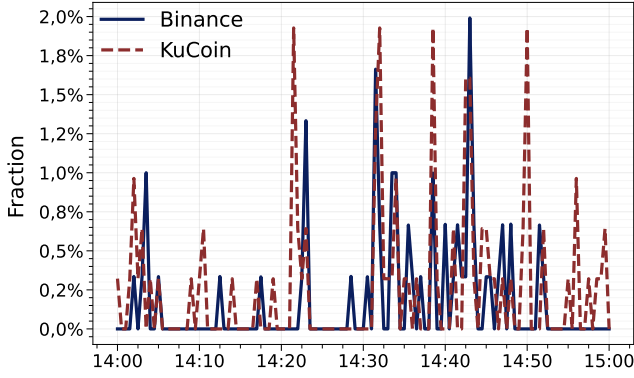


Figure 11: Fraction of active symbols in resynchronization over time, computed over successive 30 s windows.

Table 2 illustrates this contrast on two active symbols. On KuCoin, snapshot recovery is triggered mainly by sequence discontinuities. On Binance, recovery is triggered mainly when the maintained local depth limit is reached (*limit reached*). This difference is explained by the threshold itself: it is computed from the maintained book, relative to the last level of the latest snapshot. When distant levels are more tightly packed, the threshold is crossed more often and recovery is triggered more frequently. In the same window, no transport-side reordering or retry-related anomaly was observed. Output continuity is therefore interrupted by two explicit mechanisms: local depth exhaustion, which depends on the order book profile on each exchange, and upstream saturation, which propagates as explicit loss and triggers recovery.

Exchange	Symbol	Sequence breaks	Limit reached
Binance	ETH-USDT	0	19
Binance	BTC-USDT	0	54
KuCoin	ETH-USDT	21	1
KuCoin	BTC-USDT	12	3

Table 2: Illustrative recovery triggers on highly active symbols.

These figures make visible some of the mechanisms described earlier in the architectural analysis: backlog under the high regime, localized saturation during the largest bursts, explicit loss once local limits are reached, and downstream output discontinuities through recovery.

5 Conclusion

This technical note analyzed the design and operational behavior of a real-time market data pipeline under varying load conditions. It highlights the tension between preserving strong processing guarantees and maintaining bounded delay. In this system, the design prioritizes flow stability under load, at the cost of weaker guarantees under saturation. As input rates increase, backlog accumulates as latency and, once limits are reached, is converted into explicit loss and recovery. The system remains operational by keeping

saturation short and making its effects explicit. The pipeline therefore continues to produce outputs close to the current input while keeping latency under control. More generally, the analysis shows that, in this regime, stability comes from controlling how the system degrades under load. Trade-offs are made explicit, and their effects remain visible and bounded. This allows pipelines to operate continuously despite irregular and discontinuous input.

A Appendix

A.1 Feature transport and storage

This appendix describes the downstream path used to carry computed features out of the hot path and persist them for offline use. For each received order book update, the pipeline generates 6 derived features. The resulting output is therefore strongly fan-out and must be accumulated continuously through a decoupled downstream path. In this system, feature records are published by the feature extractor to a transport layer and then ingested into a feature store. Figure 12 summarizes this downstream architecture.

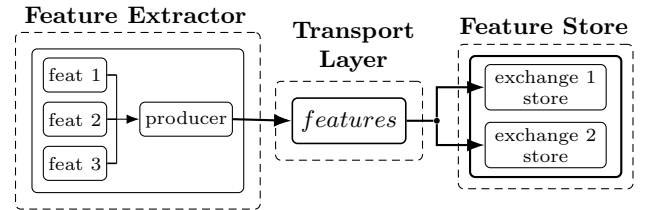


Figure 12: Downstream feature transport and storage. The example shows 3 distinct extractors, a single producer instance, and 2 exchange stores.

Downstream publication relies on Kafka, and storage relies on Apache Druid through native Kafka ingestion. Druid is used here as a distributed analytical store, with persistent data externalized to Google Cloud Storage. This provides a durable offline layer and allows downstream analysis to run independently of the current pipeline, on infrastructure sized for the required computation.

The hot path should remain non-blocking. Kafka publication does not wait for strict acknowledgements, and Druid ingestion resumes from the last committed offset. Strict end-to-end durability is therefore not enforced on this path, and rare losses may still occur under failure or network instability.

Downstream loss must remain explicit. Each feature record entering this downstream path carries an internal sequence number inherited from upstream processing. This sequence is tracked per exchange-symbol-feature stream and is reset after a crash, so any gap or backward jump directly signals downstream discontinuity. Feature records derived from the first order book emitted after a snapshot are also marked explicitly, so snapshot-induced gaps remain visible.

Downstream ingestion should also keep pace with

transport-layer retention. Otherwise, lag accumulates until old records are removed, which creates downstream loss. In the deployment described here, downstream publication runs with 6 producer tasks for KuCoin and 2 for Binance. The downstream transport layer uses 30 partitions per exchange, or about 94 exchange-symbol-feature streams per partition under an even distribution. Druid ingestion is provisioned with enough parallelism and compute capacity to stay ahead of this bound under normal load.

A.2 Snapshot request and transport

Snapshot recovery depends on an external request path that is separate from the hot update flow. The role of the snapshot middleware is to regulate snapshot requests issued by the order book engine under exchange-side API rate limits, and to return the resulting snapshots asynchronously through the transport layer. Figure 13 summarizes this architecture.

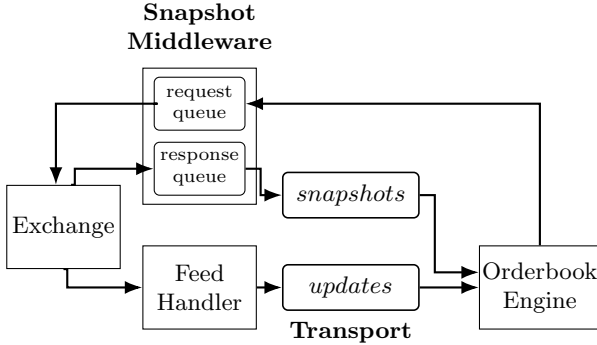


Figure 13: Snapshot request and delivery path.

Snapshot requests are first queued, then selected by a scheduler that enforces the rate limits of each exchange. The windowing policy depends on the exchange: static for KuCoin, sliding for Binance. When only a subset can be issued, selection is probabilistic and weighted toward the most active symbols. This reduces hot-path pressure without starving lower-activity symbols. Selected requests are then executed independently, and the resulting snapshots are forwarded asynchronously through the transport layer.

The snapshot middleware is designed to remain lightweight. Its state is kept in memory only, and it does not provide persistent delivery guarantees. Retry and recovery logic are therefore delegated to the caller. Once received, snapshots are published in binary form to the transport layer, where they are persisted with a replication factor of 2.

The highest load occurs at startup. Before normal processing can begin, each order book requires an initial snapshot, which creates an immediate burst of requests across all exchange-symbol pairs, about 467 per exchange in this deployment. To avoid concentrating this load at a single instant, startup is staggered: symbols are initialized in batches, with a delay of a few minutes between them.

In the deployment described here, the component is im-

plemented in Rust and runs on the `tokio` asynchronous runtime. A middleware instance is deployed per exchange. Snapshot transport uses 5 partitions and is consumed by a dedicated order book engine consumer instance for each exchange.

A.3 Deployment and observability

The system runs on a Google Kubernetes Engine cluster in the Tokyo region, with compute resources provisioned through Google Compute Engine. Most persistent storage is externalized to Google Cloud Storage. The main exception is the transport layer, which uses local SSDs on dedicated nodes for append-only log storage.

Time synchronization relies on Google NTP. Exchange clocks remain external to the system and are treated as unknown. See Appendix A.4 for details on the timestamping uncertainty bound used in the measurements.

The system is monitored continuously during operation. Each component exposes a Prometheus metrics endpoint, scraped centrally every 10 seconds and visualized in Grafana. Exported signals include CPU, memory, network usage, queue occupancy, rates, lag, drops, and saturation events. JVM-based components are also continuously profiled with Pyroscope.

A.4 Timestamping uncertainty bound

This appendix defines the inter-node timestamping uncertainty bound used in the measurements. Each node that records pipeline timestamps synchronizes independently to a common external NTP reference. Let $o_{n,k}$ denote the raw clock offset measured for node n at sampling instant k , and let $u_{n,k}$ denote the associated measurement uncertainty. For each sample, we define the node-to-reference error bound as:

$$B_{n,k} = |o_{n,k}| + u_{n,k} \quad (1)$$

Offsets were sampled throughout the experiment at intervals slightly longer than the underlying clock-update interval, with an effective sampling period of about 5 minutes. For each node n , the sampled values $\{B_{n,k}\}_k$ form an empirical distribution. Rather than using the maximum observed value, which is sensitive to isolated outliers, we summarize this distribution by its 95th percentile $B_n = q_{0.95}(\{B_{n,k}\}_k)$.

If two timestamps are recorded on nodes n_1 and n_2 , their relative timestamping error is bounded by $\varepsilon_{n_1,n_2} \leq B_{n_1} + B_{n_2}$. To obtain a single bound valid for any pair of timestamping nodes used in the experiment, we therefore define $B_{\max} = \max_n B_n$. This gives:

$$\varepsilon_{\text{inter-node}}^{\max} \leq 2B_{\max} \quad (2)$$

In the main text, this quantity is used as the timestamping-error bound for comparisons between timestamps recorded on distinct pipeline nodes. For such pairs. Timing differences below this scale cannot be interpreted reliably as pipeline-induced effects alone. Only nodes hosting timestamp measurement

points are included in this procedure. Table 3 reports the node-level bounds B_n . The largest observed value is $B_{\max} = 663 \mu s$, which gives $\varepsilon_{\text{inter-node}}^{\max} \leq 1326 \mu s$.

	1	2	3	4	5
p95 (μs)	512.0	390.0	631.0	663.0	609.0

Table 3: 95th percentile of the node-level error bound $B_{n,k} = |o_{n,k}| + u_{n,k}$ for each timestamping node over the measurement window.

A.5 Microstructure Feature Definitions

This appendix summarizes the market microstructure features computed from the reconstructed order book. These features are evaluated at each update of the local order book state produced by the processing pipeline. They rely on the best quotes and on the displayed depth across multiple price levels. Unless stated otherwise, $P_{\text{bid}}^{(l)}(t)$ and $P_{\text{ask}}^{(l)}(t)$ denote the bid and ask prices at level l of the order book at time t , while $Q_{\text{bid}}^{(l)}(t)$ and $Q_{\text{ask}}^{(l)}(t)$ denote the corresponding queue sizes.

A.5.1 Level-1 Price-Based Features

Level-1 features are derived from the best bid and ask quotes. These quantities capture the instantaneous price level and short-term price variations observed at the top of the order book.

Mid-price. The mid-price is the arithmetic mean of the best bid and best ask prices and serves as a reference price for several derived quantities:

$$m_t = \frac{P_{\text{ask}}^{(1)}(t) + P_{\text{bid}}^{(1)}(t)}{2} \quad (3)$$

Micro-price. The micro-price adjusts the mid-price by weighting the best bid and ask prices according to the available queue sizes on the opposite side. It provides a liquidity-weighted estimate of the short-term equilibrium price:

$$P_t^{\text{micro}} = \frac{P_{\text{ask}}^{(1)}(t) Q_{\text{bid}}^{(1)}(t) + P_{\text{bid}}^{(1)}(t) Q_{\text{ask}}^{(1)}(t)}{Q_{\text{bid}}^{(1)}(t) + Q_{\text{ask}}^{(1)}(t)} \quad (4)$$

Mid-price log return. For completeness, we recall the mid-price log return already introduced in the main text. It measures the logarithmic change in mid-price between two consecutive updates:

$$r_t = \log m_t - \log m_{t^-} \quad (5)$$

where t^- denotes the most recent timestamp strictly preceding t .

Spread. The bid-ask spread measures the instantaneous price difference between the best ask and best bid quotes:

$$s_t = P_{\text{ask}}^{(1)}(t) - P_{\text{bid}}^{(1)}(t), \quad \Delta s_t = s_t - s_{t^-} \quad (6)$$

It reflects the tightness of the order book and is often interpreted as a proxy for short-term liquidity conditions.

A.5.2 Level-2 Depth and Imbalance Features

Level-2 features incorporate information from several levels of the order book and capture the distribution of liquidity beyond the best quotes.

Multi-level imbalance. The multi-level imbalance compares the cumulative bid and ask depth across the first K levels of the order book:

$$I_K(t) = \frac{\sum_{l=1}^K Q_{\text{bid},l}(t) - \sum_{l=1}^K Q_{\text{ask},l}(t)}{\sum_{l=1}^K Q_{\text{bid},l}(t) + \sum_{l=1}^K Q_{\text{ask},l}(t)} \quad (7)$$

Positive values indicate stronger bid-side liquidity, while negative values indicate stronger ask-side liquidity.

Depth in a τ -bps band. The displayed depth within a symmetric τ -basis-point band around the mid-price is computed as

$$D_\tau(t) = \sum_{l: P_{\text{bid}}^{(l)}(t) \geq (1-\tau)m_t} Q_{\text{bid},l}(t) + \sum_{l: P_{\text{ask}}^{(l)}(t) \leq (1+\tau)m_t} Q_{\text{ask},l}(t) \quad (8)$$

This quantity measures the total liquidity available close to the current trading price.

A.6 Additional observations

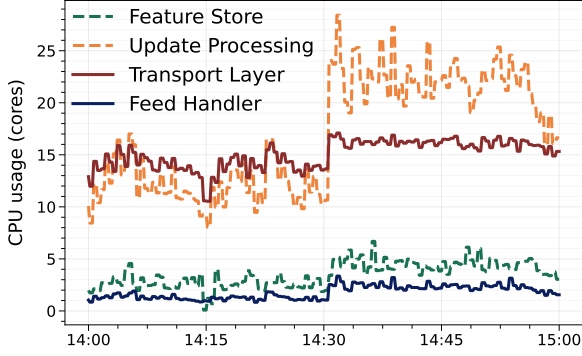


Figure 14: CPU usage for each layer of the hot path, computed over successive 10s windows.

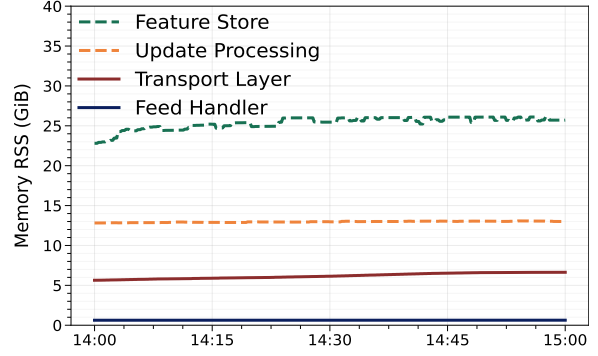


Figure 15: Memory usage for each layer of the hot path, computed over successive 10s windows.

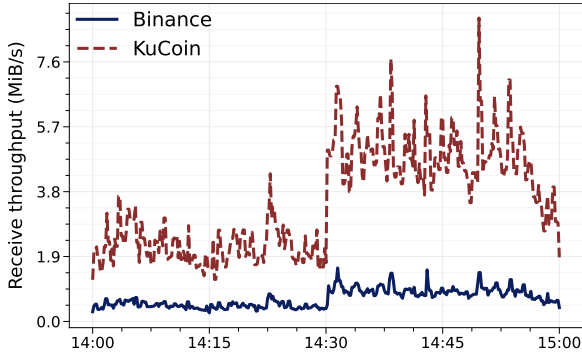


Figure 16: Feed handler network receive rate (MiB/s) for Binance and KuCoin, computed over successive 30s windows.

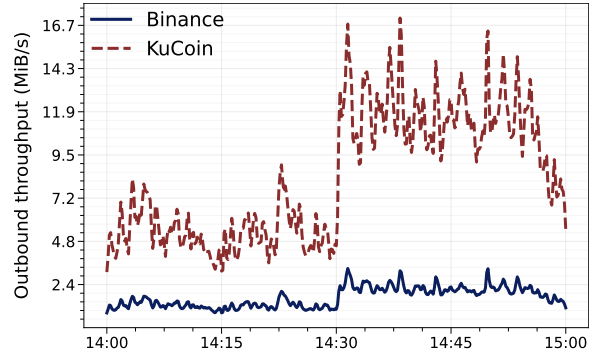


Figure 17: Feature output volume (MiB/s) entering Transport Layer II at the end of the hot path for Binance and KuCoin, in successive 30s windows.

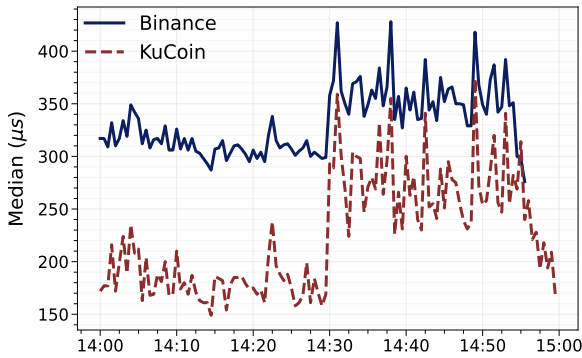


Figure 18: Median across symbols of the per-symbol median per-event maximum inter-feature delay (μs), computed over successive 30s windows.

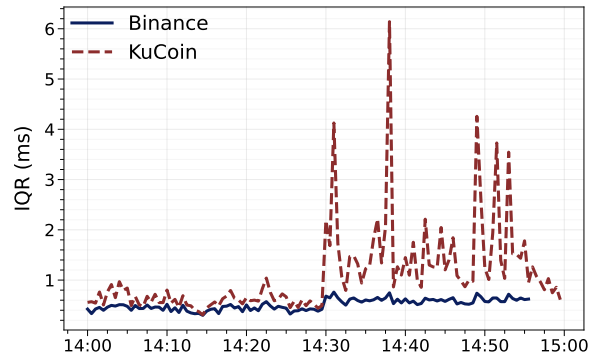


Figure 19: Cross-symbol lag dispersion over time, computed on the mid-price output as the IQR of per-symbol median lag, in successive 30s windows.

References

- [1] Binance. Binance spot api documentation. <https://developers.binance.com/docs/binance-spot-api-docs>, 2026. Accessed during the experiments.
- [2] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
- [3] Charles Colella. On the temporal structure of distributed pipelines. Technical note No 2, May, 2026.
- [4] Charles Colella. Real-time market data system. <https://github.com/charlescol/real-time-market-data-system>, 2026. GitHub repository.
- [5] Jay Kreps, Neha Narkhede, Jun Rao, et al. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, volume 11, pages 1–7. Athens, Greece, 2011.
- [6] KuCoin. Kucoin api documentation. <https://www.kucoin.com/docs-new/introduction>, 2026. Accessed during the experiments.